

IT Helpdesk Ticket Classifier

Project Report

Natural Language Processing | Text Classification | Machine Learning | Streamlit Deployment

1. Project Overview

This project builds a machine learning model that reads IT support ticket text and automatically classifies it into one of eight predefined categories. The goal is to reduce the manual effort required by helpdesk staff to sort and route incoming tickets.

In a real IT department, hundreds of tickets arrive every day. Manually reading each one and deciding which team should handle it takes time and is prone to human error. An automated classifier can do this instantly and consistently.

The project also includes a fully functional web application built with Streamlit that allows users to classify single tickets or upload a CSV file for bulk classification with downloadable results.

The eight ticket categories used in this project are:

- Hardware
- HR Support
- Access
- Miscellaneous
- Storage
- Purchase
- Internal Project
- Administrative rights

2. Dataset

2.1 Source

Dataset Name: IT Service Ticket Classification Dataset

Source: Kaggle — <https://www.kaggle.com/datasets/adisongoh/it-service-ticket-classification-dataset>

File: all_tickets_processed_improved_v3.csv

2.2 Structure

The dataset contains 47,837 rows and 2 columns.

Column	Type	Description
Document	Text	The full text of the IT support ticket
Topic_group	Text	The category label assigned to the ticket

2.3 Category Distribution

The dataset is imbalanced. Hardware has the most tickets and Administrative rights has the fewest.

Category	Ticket Count
Hardware	13,617
HR Support	10,915
Access	7,125
Miscellaneous	7,060
Storage	2,777
Purchase	2,464
Internal Project	2,119
Administrative rights	1,760

3. Tools and Libraries

Library	Purpose
Python 3	Programming language used throughout the project
Pandas	Loading and exploring the dataset
Scikit-learn	TF-IDF vectorization, model training, and evaluation
Matplotlib	Plotting charts and visualizations
Seaborn	Plotting the confusion matrix heatmap
Streamlit	Building and running the web application
streamlit-option-menu	Horizontal navigation menu in the app
Pickle	Saving and loading the trained model and vectorizer
Google Colab	Model training environment
PyCharm	Local development environment for the Streamlit app

4. Methodology

4.1 Text Cleaning

The raw ticket text was cleaned using the following steps:

- Converted all text to lowercase
- Removed special characters, numbers, and punctuation
- Removed extra whitespace

This ensures the model focuses on meaningful words rather than noise.

4.2 TF-IDF Vectorization

TF-IDF stands for Term Frequency - Inverse Document Frequency. It converts text into numerical values that a machine learning model can understand.

- Words that appear often in one ticket but rarely across all tickets get a high score
- Common words like the, is, and are automatically given a low score
- The top 10,000 words were selected as features
- Standard English stop words were removed

4.3 Train and Test Split

The data was split into two parts before training the model:

Split	Percentage	Number of Samples
Training Set	80%	38,269
Testing Set	20%	9,568

4.4 Model — Logistic Regression

Logistic Regression was selected as the classifier. It is a well-established algorithm for multi-class text classification problems. It is fast to train, easy to interpret, and performs well on TF-IDF features.

- `max_iter` was set to 1000 to allow the model enough steps to converge
- `random_state` was set to 42 for reproducibility
- Training completed in 7.39 seconds on Google Colab

4.5 Model Saving

After training, the model and vectorizer were saved as pickle files so they can be loaded by the Streamlit application without retraining.

File	Contents
model.pkl	Trained Logistic Regression classifier
tfidf.pkl	Fitted TF-IDF vectorizer with 10,000 word vocabulary

5. Results

5.1 Overall Accuracy

84.53%

The model correctly classified 84.53% of tickets in the test set, which it had never seen during training. This is a strong result for a baseline model using only Logistic Regression.

5.2 Per-Category Results

Category	Precision	Recall	F1 Score	Test Samples
Access	0.92	0.87	0.89	1,455
Administrative rights	0.87	0.68	0.76	342
HR Support	0.85	0.83	0.84	2,107
Hardware	0.79	0.88	0.83	2,760
Internal Project	0.91	0.80	0.86	451
Miscellaneous	0.80	0.82	0.81	1,400
Purchase	0.97	0.88	0.92	497
Storage	0.93	0.84	0.88	556
Overall (weighted avg)	0.85	0.85	0.85	9,568

5.3 Key Observations

- Purchase achieved the highest precision at 0.97, meaning nearly all tickets predicted as Purchase were correct
- Access and Storage also performed very well with F1 scores above 0.88
- Administrative rights had the lowest recall at 0.68, likely because it has the fewest training samples at 1,760
- Hardware and HR Support showed some confusion with each other, which is expected as both categories can contain similar language around employee requests

6. Custom Ticket Prediction

The trained model was tested on five custom ticket descriptions written manually to verify it works on real-world inputs.

Ticket Text	Predicted Category	Correct
My laptop screen is broken and I cannot turn it on	Hardware	Yes
I need access to the HR portal to submit my leave request	HR Support	Yes
Please give me admin rights to install software on my computer	Administrative rights	Yes
My external hard drive is not showing up on the network storage	Storage	Yes
I want to purchase a new keyboard and mouse for my workstation	Hardware	Partial

4 out of 5 predictions were fully correct. The fifth ticket about purchasing a keyboard and mouse was predicted as Hardware instead of Purchase. This is an understandable mistake because the ticket mentions specific hardware items, which the model associated with the Hardware category.

7. Streamlit Web Application

7.1 Overview

The trained model was deployed as a local web application using Streamlit. The app allows anyone to use the classifier without writing any code. It runs locally in PyCharm using a Python virtual environment.

7.2 Project Structure

File / Folder	Purpose
app.py	Main Streamlit application file
model/model.pkl	Saved Logistic Regression classifier
model/tfidf.pkl	Saved TF-IDF vectorizer
requirements.txt	List of all required Python libraries
README.md	Project documentation for GitHub

7.3 App Features

The application has three tabs accessible from a horizontal navigation menu at the top.

Tab	Description
Single Ticket	User types a ticket description and gets the predicted category with a confidence score
CSV Upload	User uploads a CSV file with a Document column and gets all tickets classified in bulk with a downloadable results file
About the Model	Displays model information, training details, and per-category performance metrics

7.4 How to Run Locally

Follow these steps to run the application on any machine:

- Create a project folder named IT_Helpdesk_Classifier
- Place model.pkl and tfidf.pkl inside a subfolder named model
- Open the folder in PyCharm and create a virtual environment using `python -m venv venv`
- Activate the virtual environment and run `pip install -r requirements.txt`
- Run the app with `streamlit run app.py`
- The app will open automatically in the browser at `http://localhost:8501`

7.5 Libraries Used in the App

Library	Purpose
streamlit	Web application framework
streamlit-option-menu	Horizontal tab navigation
scikit-learn	Loading model and vectorizer, making predictions
pandas	Handling CSV uploads and results
pickle	Deserializing the saved model and vectorizer
re	Text cleaning with regular expressions

8. Conclusion

This project successfully built and deployed an IT helpdesk ticket classifier using natural language processing and machine learning. The model achieved 84.53% accuracy on unseen test data using a straightforward pipeline of text cleaning, TF-IDF vectorization, and Logistic Regression.

The project was then taken a step further by deploying it as a fully functional Streamlit web application that supports both single ticket classification and bulk CSV processing. The app runs locally and is ready to be extended or deployed to a cloud platform.

Future improvements could include:

- Using more advanced models such as Random Forest or BERT for higher accuracy
- Applying oversampling techniques to handle the class imbalance in smaller categories
- Adding a confidence threshold to flag low-confidence predictions for manual review
- Deploying the app to Streamlit Cloud or a cloud service for public access

Dataset: Kaggle — [adisongoh/it-service-ticket-classification-dataset](#) | Training: Google Colab | Deployment: Streamlit + PyCharm